

Experiment No 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- Automation: Executes actions automatically when conditions are satisfied.
- Transparency: The contract code and transactions are visible on the blockchain.
- Security: Once deployed, contracts are immutable and tamper-proof.
- Cost-effective: Removes intermediaries and reduces transaction costs.
- Trustless System: Participants can interact without needing to trust each other directly. Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Code:\

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract PharmaSupply {

    address public manufacturer;
    address public pharmacy;

    struct Medicine
    { string name;
      uint quantity;
      bool shipped;
      bool received;
    }
    Medicine public med;
    constructor(address _pharmacy) {
        manufacturer = msg.sender;
        pharmacy = _pharmacy;
    }
}
```

```

    }

    function addMedicine(string memory _name, uint _quantity) public {
        require(msg.sender == manufacturer, "Only manufacturer allowed"); med
        = Medicine(_name, _quantity, false, false);
    }

    function shipMedicine() public {
        require(msg.sender == manufacturer, "Only manufacturer allowed");
        med.shipped = true;
    }

    function receiveMedicine() public {
        require(msg.sender == pharmacy, "Only pharmacy allowed");
        require(med.shipped == true, "Not shipped yet"); med.received
        = true;
    }

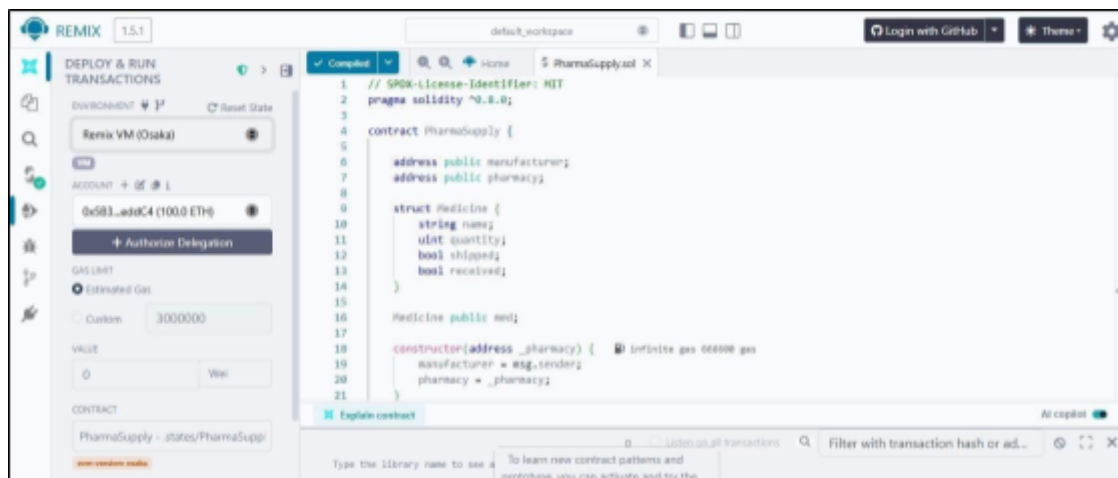
    function getMedicine() public view returns (string memory, uint, bool, bool) { return
        (med.name, med.quantity, med.shipped, med.received);
    }
}

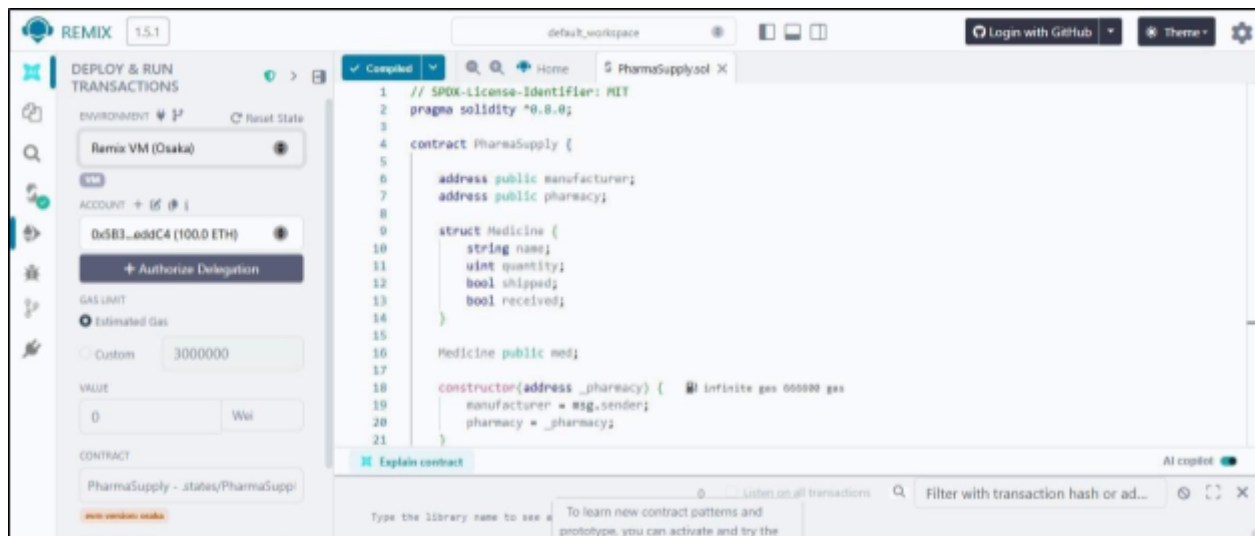
```

Implementation:

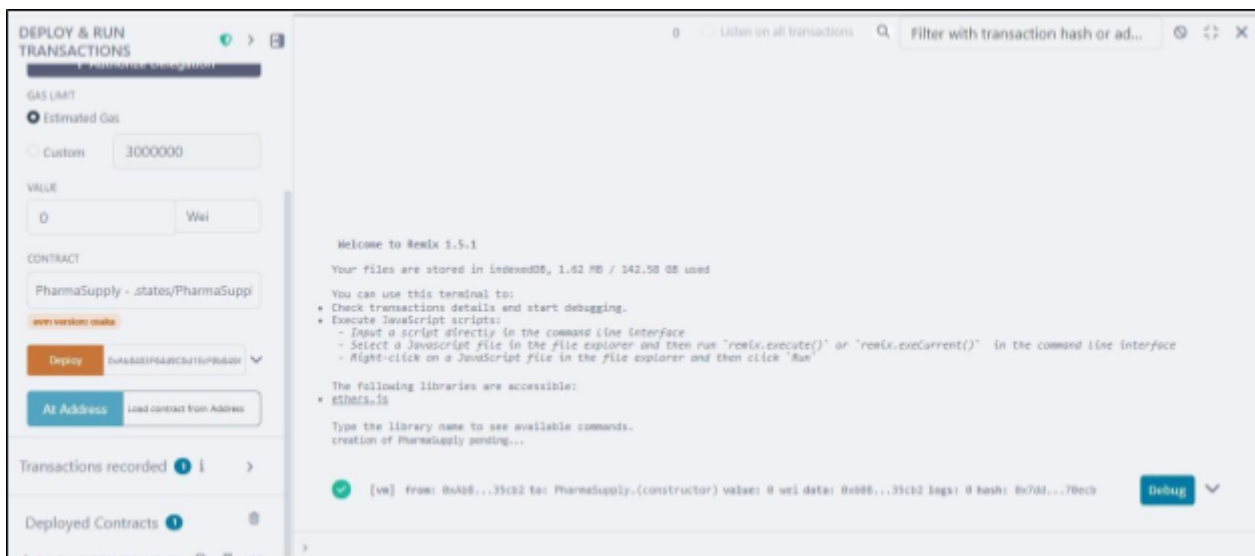
Contract 1: PharmaSupply (Medicine flow)

Contract code

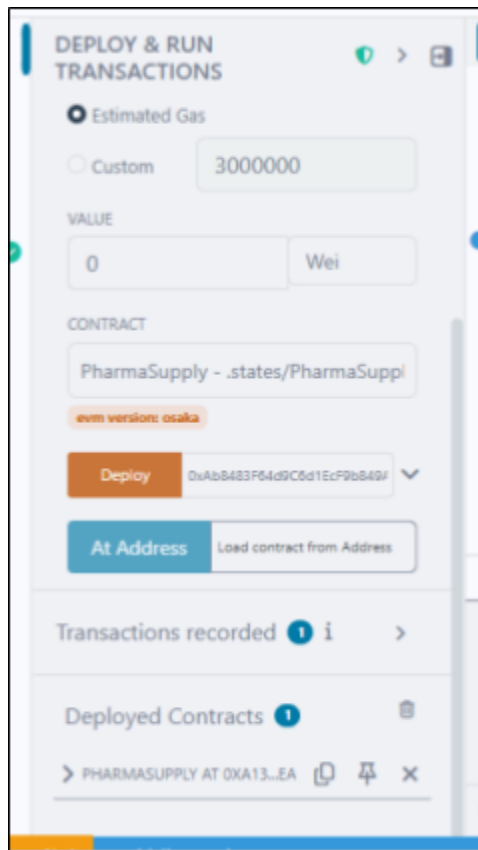




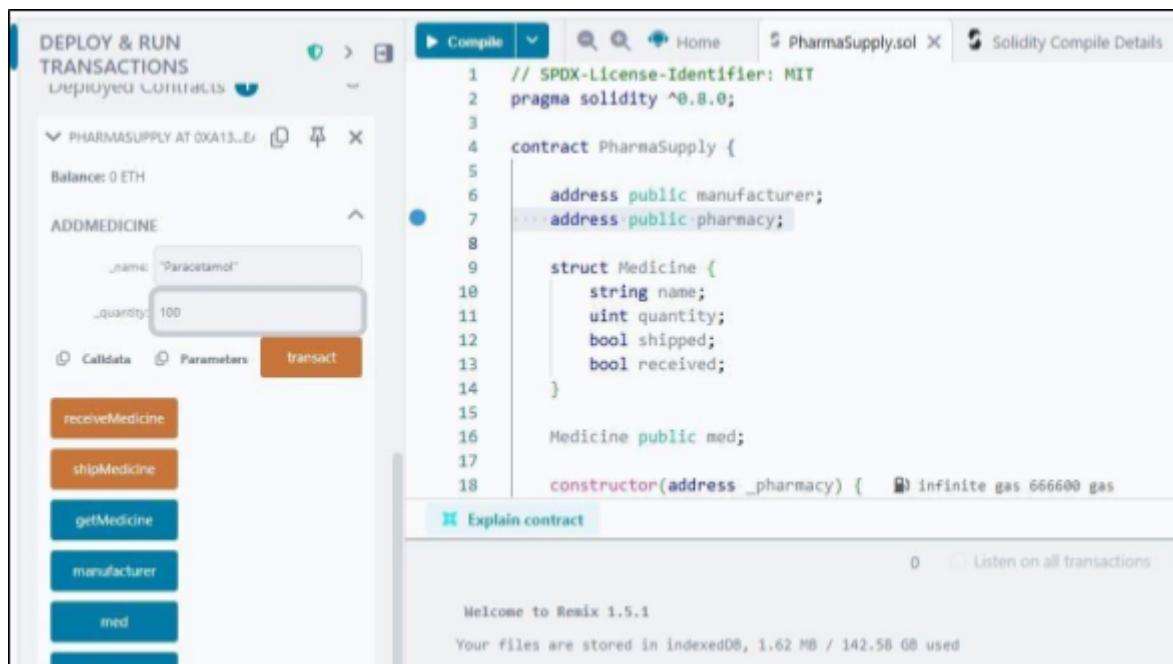
Transaction deployed successfully



Deployed contract



Manufacture adding medicine for shipping



Manufacturer successfully added medicine



DEPLOY & RUN TRANSACTIONS

PHARMASUPPLY AT 0xA13...E

Balance: 0 ETH

ADD MEDICINE

name: "Paracetamol"

quantity: 100

CallData Parameters **Interact**

executeMedicine

shipMedicine

getMedicine

manufacturer

red

pharmacy

Low level interactions

Listen on all transactions Filter with transaction hash or ad...

[vm] from: 0x583...addC4 to: PharmaSupply.addMedicine(string,uint256) 0x13...eAD95 value: 0 wei data: 0x098...00000 logs: 0 hash: 0xf9a...47e9a

transact to PharmaSupply.addMedicine errored: error occurred: revert.

revert

The transaction has been reverted to the initial state.
Reason provided by the contract: "Only manufacturer allowed".
If the transaction failed for not having enough gas, try increasing the gas limit gently.

[vm] from: 0x583...addC4 to: PharmaSupply.addMedicine(string,uint256) 0x13...eAD95 value: 0 wei data: 0x098...00000 logs: 0 hash: 0xf9a...47e9a

transact to PharmaSupply.addMedicine errored: error occurred: revert.

revert

The transaction has been reverted to the initial state.
Reason provided by the contract: "Only manufacturer allowed".
If the transaction failed for not having enough gas, try increasing the gas limit gently.

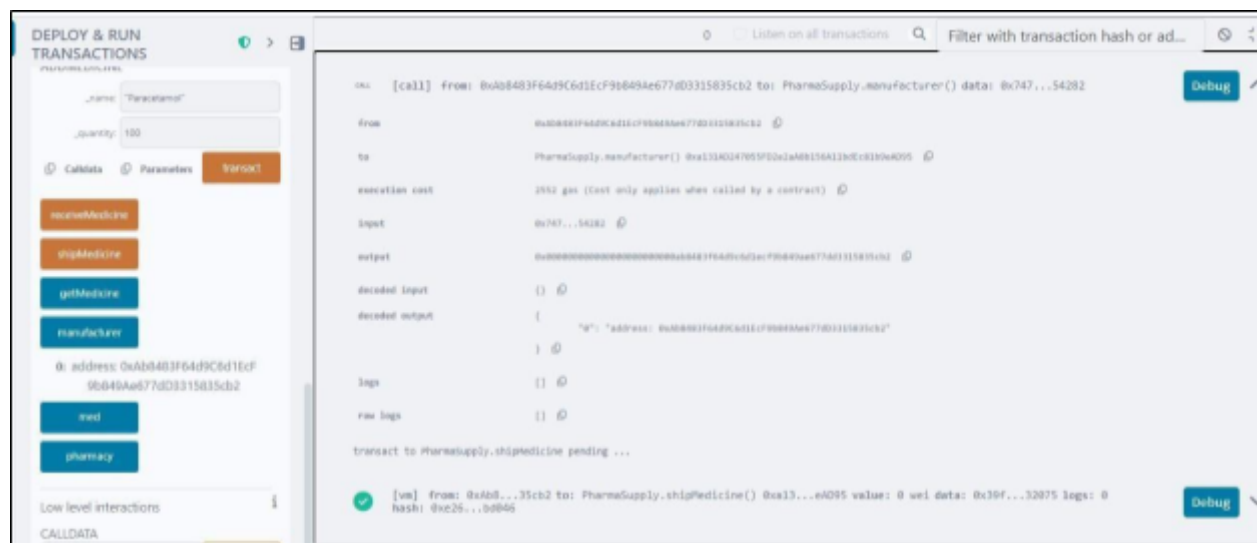
transact to PharmaSupply.addMedicine pending ...

transact to PharmaSupply.addMedicine pending ...

[vm] from: 0x0a8...35cb2 to: PharmaSupply.addMedicine(string,uint256) 0x13...eAD95 value: 0 wei data: 0x098...00000 logs: 0 hash: 0xb32...45d08

[vm] from: 0x0a8...35cb2 to: PharmaSupply.addMedicine(string,uint256) 0x13...eAD95 value: 0 wei data: 0x098...00000 logs: 0 hash: 0xb32...45d08

Manufacturer Shipping the medicine



DEPLOY & RUN TRANSACTIONS

PHARMASUPPLY AT 0xA13...E

name: "Paracetamol"

quantity: 100

CallData Parameters **Interact**

executeMedicine

shipMedicine

getMedicine

manufacturer

red

pharmacy

0: address: 0xA6483F64D9C6D1EcF9644bAe77dD3315835cb2

Low level interactions

CALLDATA

Listen on all transactions Filter with transaction hash or ad...

OK [call] from: 0xA6483F64D9C6D1EcF9644bAe77dD3315835cb2 to: PharmaSupply.manufacturer() data: 0x747...54282

from: 0xA6483F64D9C6D1EcF9644bAe77dD3315835cb2

to: PharmaSupply.manufacturer() 0xA13A024765F00a08136A130E0180A005

execution cost: 2052 gas (Cost only applies when called by a contract)

input: 0x747...54282

output: 0x00

decoded input: {}

decoded output: {"a": "address: 0xA6483F64D9C6D1EcF9644bAe77dD3315835cb2"}

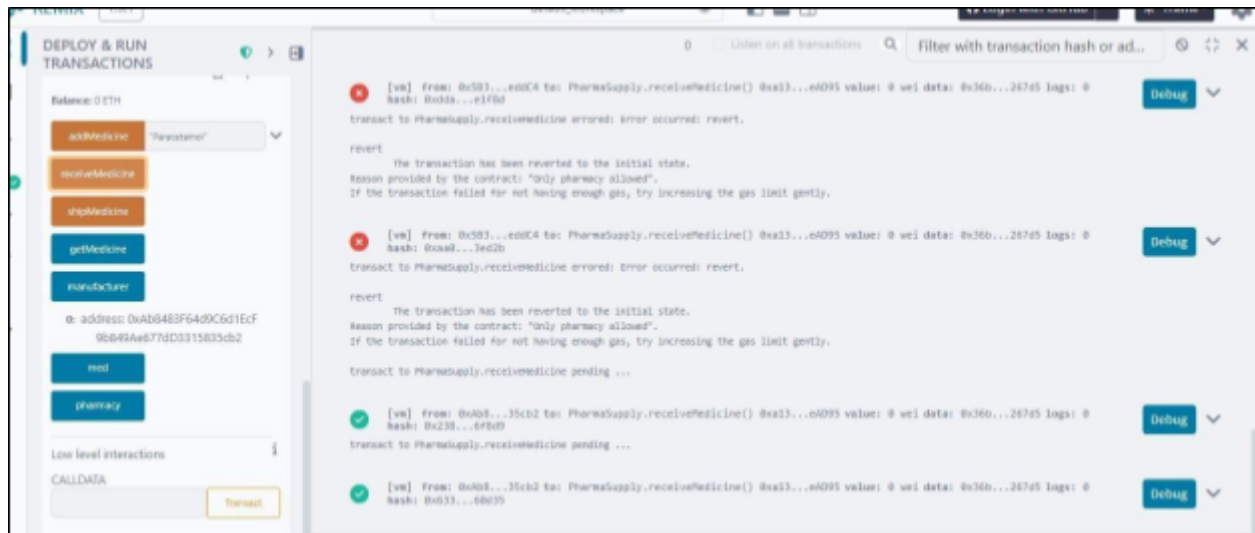
logs: []

raw logs: []

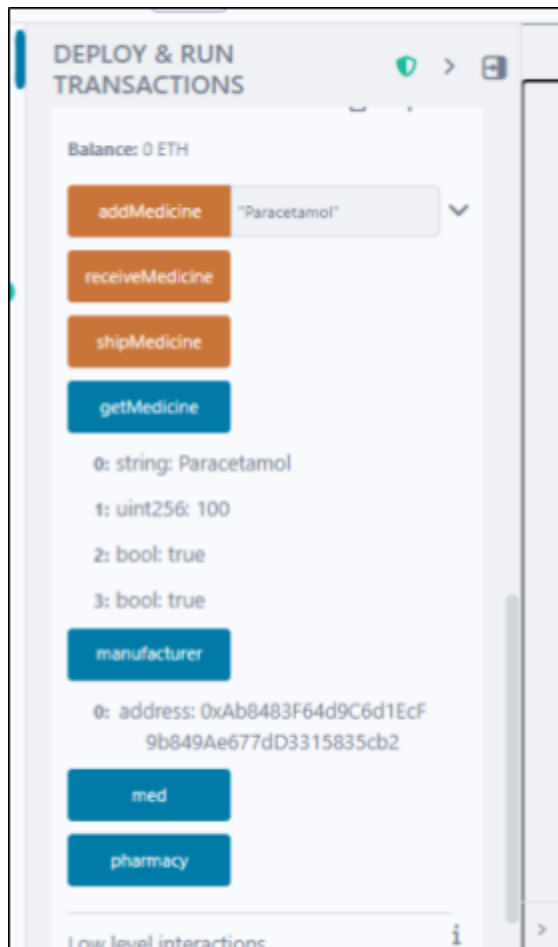
transact to PharmaSupply.shipMedicine pending ...

[vm] from: 0xb0d...35cb2 to: PharmaSupply.shipMedicine() 0x13...eAD95 value: 0 wei data: 0x10f...32075 logs: 0 hash: 0xe28...b0846

Medicine received by pharmacy successfully



getmedicine() we are getting the values of added medicine



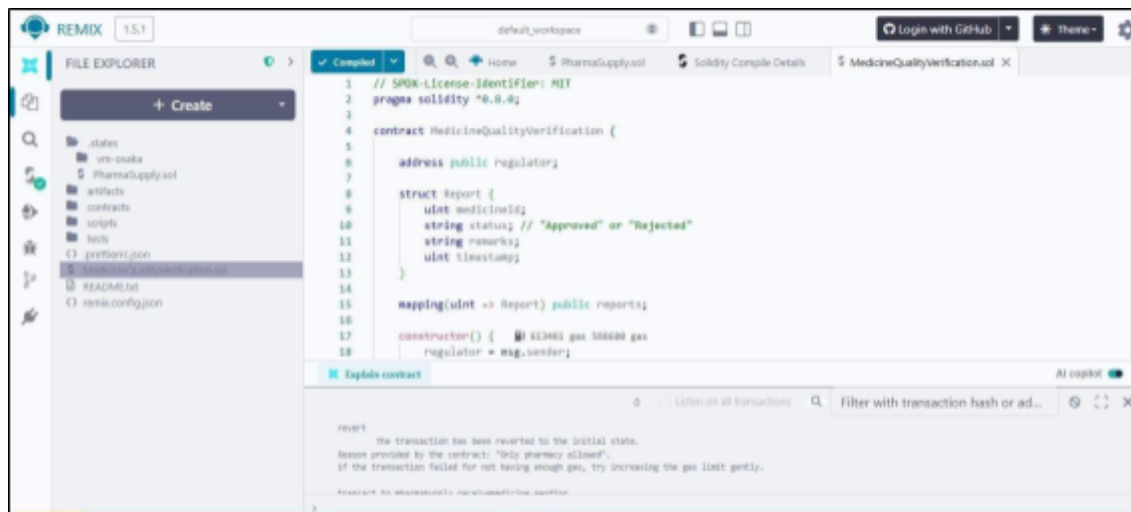
Contract 2: medicine quality verification

While verifying:

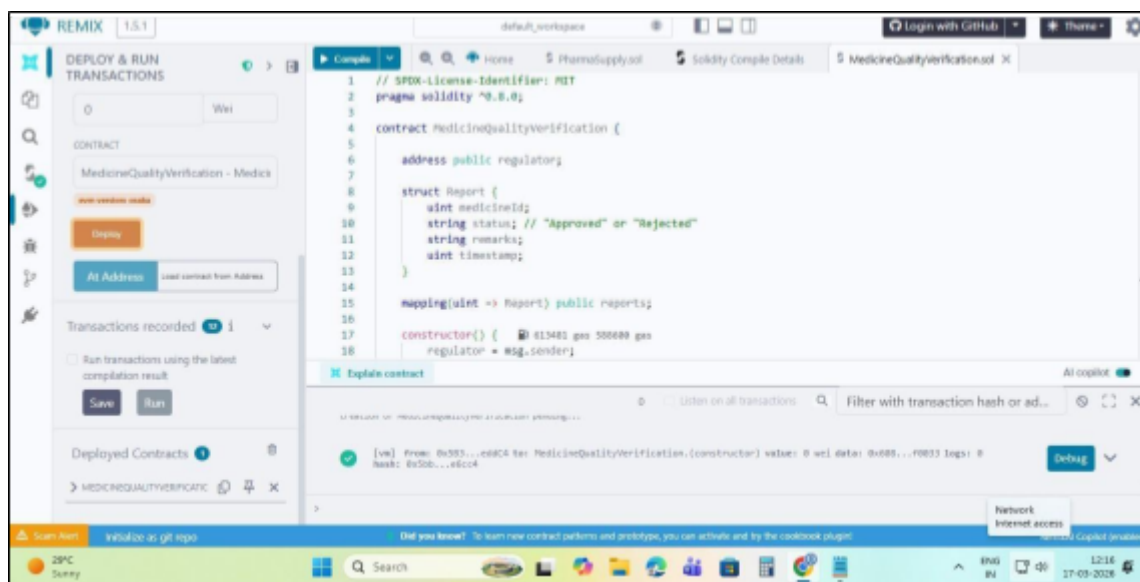
“Here, the regulator verifies the medicine batch and stores its approval status on blockchain.”

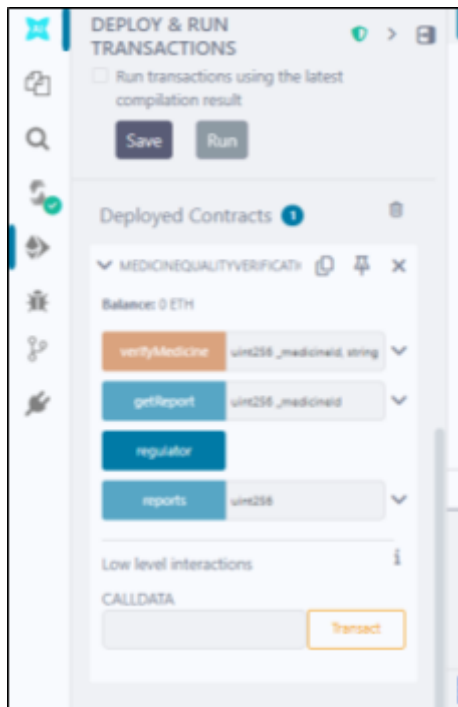
While fetching:

“This function retrieves the verification report, ensuring transparency and immutability.”

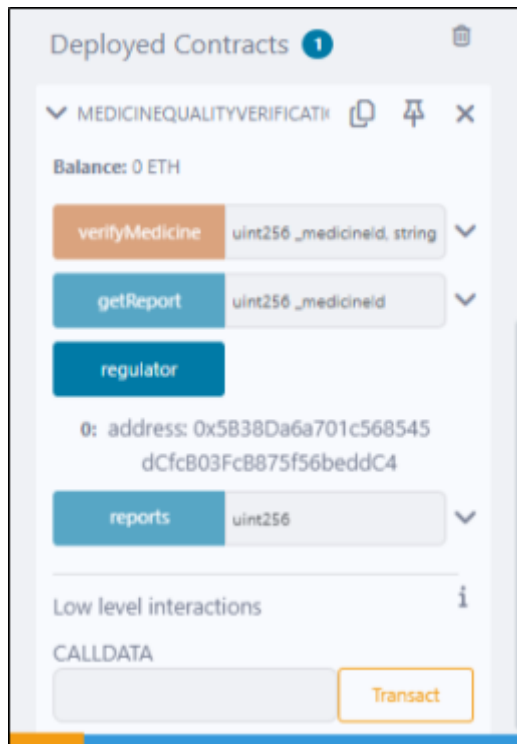


Contract deployed

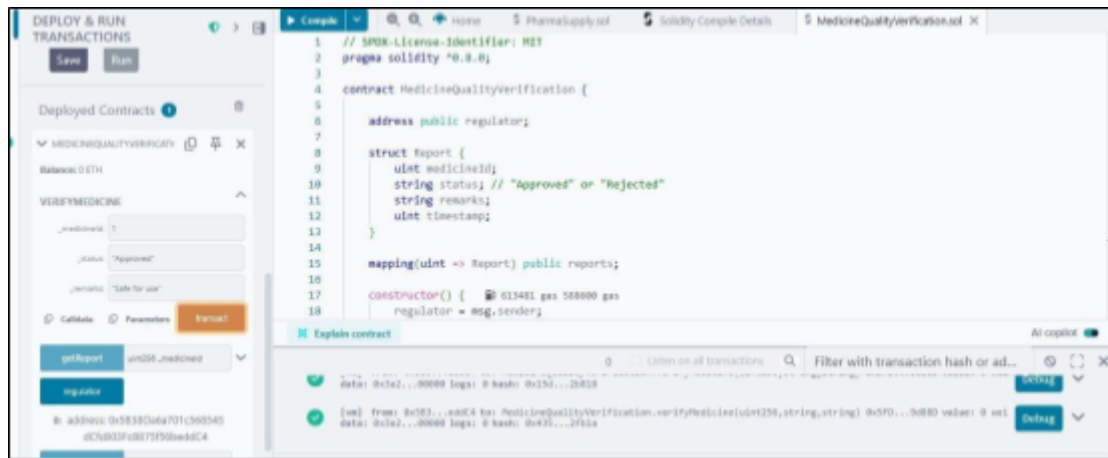




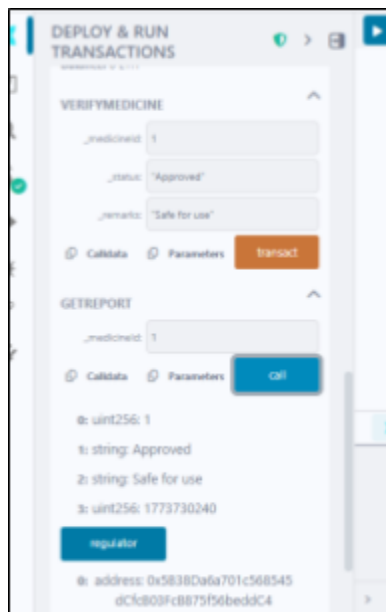
Regulator



Medicine verification done by regulator



getreport

**Conclusion:**

The experiment was successfully completed using Remix IDE, where two Solidity smart contracts were deployed and tested on the Ethereum blockchain. Contract 1 (PharmaSupply) demonstrated the medicine flow from manufacturer to pharmacy, while Contract 2 (MedicineQualityVerification) allowed a regulator to verify and store quality reports on-chain. Role-based access control was enforced using `require` statements, and all transactions were recorded immutably. The experiment successfully demonstrated the use of smart contracts for transparent and trustless pharmaceutical supply chain management.